CS480P/580P Introduction to Natural Language Processing

Patrick H. Chen

October 16, 2025

Lecture 15

Standard Prompting

```
# K-shot in-context exemplars
Q: {question}
A: {answer}

Q: {question}
A: {answer}

...
# One sample to inference
Q: Ali had $21. Leila gave him half of her
   $100. How much does Ali have now?
----------------------------------------
# Response
A: Leila gave 100/2=50 to Ali. Ali now has
   $21+$50 = $71. The answer is 71.
```

Batch Prompting

```
# K-shot in-context exemplars in K/b batches
Q[1]: {question}
Q[2]: {question}
A[1]: {answer}
A[2]: {answer}

...
# b samples in a batch to inference
Q[1]: Ali had $21. Leila gave him half of her
      $100. How much does Ali have now?
Q[2]: A robe takes 2 bolts of blue fiber and
      half that white fiber. How many bolts?
----------------------------------------------
# Responses to a batch
A[1]: Leila gave 100/2=50 to Ali. Ali now has
      $21+$50 = $71. The answer is 71.
A[2]: It takes 2/2=1 bolt of white fiber. The
      total amount is 2+1=3. The answer is 3.
```

b(=2) samples in one batch

Algorithm 1 Two-Stage DPP Selection

Input: Training set $\mathcal{D} = \{e_i\}_{i=1}^{N}$, language model LM, candidate subset size N_{sem}, score set size T, demonstration set size k, sentence encoder sbert.

Output: Demonstration Subset $\mathcal{D}_{\text{dem}}$

for $e_i \in \mathcal{D}$ **do**
  $\boldsymbol{x}_i = \text{sbert}(x_i)$
end for
$\boldsymbol{s} = \text{stack}(\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N)$
$\boldsymbol{L}_S = \boldsymbol{s}\boldsymbol{s}^T$
$\mathcal{D}_{\text{sem}} = \arg\max_{Y \subseteq \mathcal{D}, |Y| = N_{\text{sem}}} \frac{\det(\boldsymbol{L}_Y)}{\det(\boldsymbol{L}_S + \boldsymbol{I})}$
random sampling $\mathcal{D}_{\text{score}}$ of size T from $\mathcal{D}/\mathcal{D}_{\text{sem}}$
for $e_i \in \mathcal{D}_{\text{sem}}$ **do**
  for $e_j \in \mathcal{D}_{\text{score}}$ **do**
    $\text{Inf}(e_i, e_j) = p_{\text{LM}}(y_j | e_i, x_j) - p_{\text{LM}}(y_j | x_j)$
  end for
  $\text{Q}(e_i) = \frac{\sum_{e_j \in \mathcal{D}_{\text{score}}} \textbf{Inf}(e_i, e_j)}{T}$
  $\boldsymbol{e_i} = (\text{Inf}(e_i, e_1), \text{Inf}(e_i, e_2), \cdots, \text{Inf}(e_i, e_T))$
end for
$\boldsymbol{I} = \text{stack}(\boldsymbol{e}_1, \boldsymbol{e}_2, \cdots, \boldsymbol{e}_{N_{\text{sem}}})$
$\boldsymbol{Q} = (\text{Q}_{e_1}, \text{Q}_{e_2}, \cdots, \text{Q}_{e_{N_{\text{sem}}}})$
$\boldsymbol{L}_I = \boldsymbol{Q} \cdot \boldsymbol{I}\boldsymbol{I}^T \cdot \boldsymbol{Q}$
$\mathcal{D}_{\text{dem}} = \arg\max_{Y \subseteq \mathcal{D}_{\text{sem}}, |Y| = k} \frac{\det(\boldsymbol{L}_Y)}{\det(\boldsymbol{L}_I + \boldsymbol{I})}$
return $\mathcal{D}_{\text{dem}}$

Model	Method	SST-2	Trec	CB	AGNews	DBpedia	RTE	AVG
		2/4/8	2/4/8	2/4/8	2/4/8	2/4/8	2/4/8	2/4/8
GPT2-xl 1.5B	LC	52.14/ 53.72/55.86	18.20/22.40/36.20	43.78/44.85/46.75	37.62/38.48/41.27	39.45/46.74/52.08	48.95/51.02/50.24	40.02/42.86/47.06
	Cal	53.88/56.04/59.17	19.10/24.30//38.10	45.86/46.55/46.62	41.02/42.58/45.12	54.36/61.32/69.56	50.22/49.35/50.89	44.07/46.70/51.58
	Random	52.33/55.27/56.97	17.36/23.76/37.20	46.43/46.47/46.57	40.50/41.86/43.84	55.80/61.24/68.06	51.84/52.20/52.56	44.04/46.80/50.87
	Cluster	53.08/55.43/60.37	18.34/26.40/44.97	47.38/48.76/52.15	63.57/64.89/68.02	59.33/67.28/75.47	51.33/50.96/51.76	48.84/50.29/58.79
	DPP	53.02/56.12/63.59	18.20/27.60/50.80	47.07/50.00/55.36	63.88/66.72/69.86	58.48/69.44/78.77	51.57/51.01/53.43	48.70/53.48/61.97
	Ours	59.86/64.31/72.40	32.68/38.60/57.22	48.21/56.07/59.14	67.21/73.91/76.42	65.93/67.74/75.35	53.07/53.79/53.77	54.49/59.07/65.72
GPT-J 6B	LC	81.87/84.76/88.02	34.20/37.10/40.60	24.45/50.27/53.87	53.87/54.98/66.04	60.96/76.65/80.47	50.07/51.88/52.24	50.90/59.27/63.54
	Cal	84.35/89.04/89.57	37.02/38.86/43.76	27.98/54.55/58.04	56.65/58.72/69.27	64.43/81.69/83.58	52.02/52.96/53.94	53.74/62.63/66.36
	Random	84.20/88.57/89.06	36.80/38.08/44.40	25.36/53.57/55.36	55.74/56.80/67.19	62.23/80.46/81.44	55.02/56.31/53.86	53.23/62.30/65.22
	Cluster	85.34/88.96/90.90	37.22/39.50/46.26	33.24/52.87/58.15	64.88/67.83/71.04	68.44/83.52/84.93	51.88/52.45/54.02	56.83/64.19/67.55
	DPP	86.32/89.67/90.81	38.40/40.60/53.40	47.47/56.10/62.07	74.26/76.30/80.54	75.03/86.28/88.95	52.65/54.90/63.18	62.36/67.31/73.16
	Ours	88.20/91.67/92.61	58.02/67.60/72.78	49.50/58.66/64.29	79.66/82.26/84.09	80.45/89.26/92.47	53.02/55.48/56.63	68.14/74.16/77.15
GPT-NeoX 20B	LC	82.55/85.47/89.14	42.18/51.53/60.89	46.99/58.74/60.02	69.43/72.29/75.86	70.08/78.24/83.04	54.82/55.67/57.88	61.01/66.99/71.14
	Cal	84.40/90.01/93.22	44.72/54.88/61.97	50.05/61.37/62.18	73.62/75.47/79.15	74.20/83.09/84.22	56.44/57.05/57.92	63.91/70.31/73.11
	Random	83.57/89.25/92.89	44.56/53.80/63.12	49.13/60.71/60.00	74.13/73.73/77.91	72.66/81.84/83.68	57.26/59.06/57.83	63.55/69.73/72.57
	Cluster	84.48/90.39/93.02	44.88/55.12/63.87	51.18/61.27/62.05	75.89/76.78/80.42	74.05/82.56/84.32	57.40/58.02/58.34	64.65/70.69/73.67
	DPP	86.59/93.19/93.41	46.00/57.20/65.40	50.00/62.93/63.29	77.36/80.34/83.43	80.76/89.32/91.08	58.48/56.48/58.87	66.53/73.24/75.91
	Ours	88.76/93.80/94.48	61.12/71.03/79.94	54.18/65.56/69.27	81.57/84.76/86.48	84.14/92.27/93.62	58.43/58.87/59.22	71.37/77.72/80.50

Eliciting Outputs

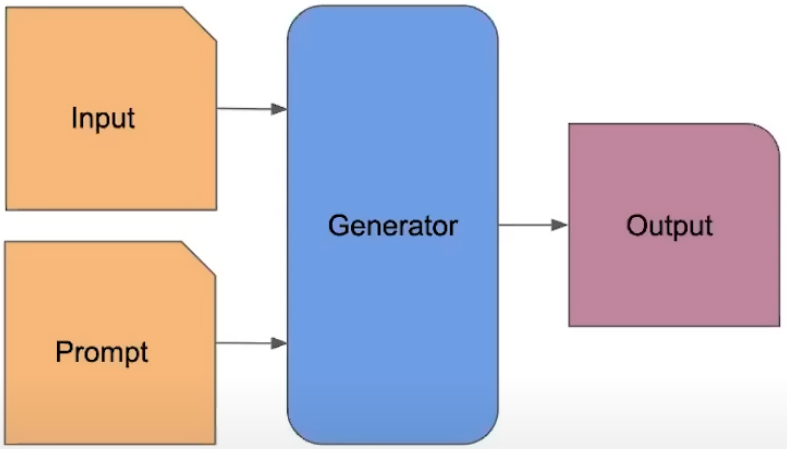
So we elicit specific outputs.

Problems:

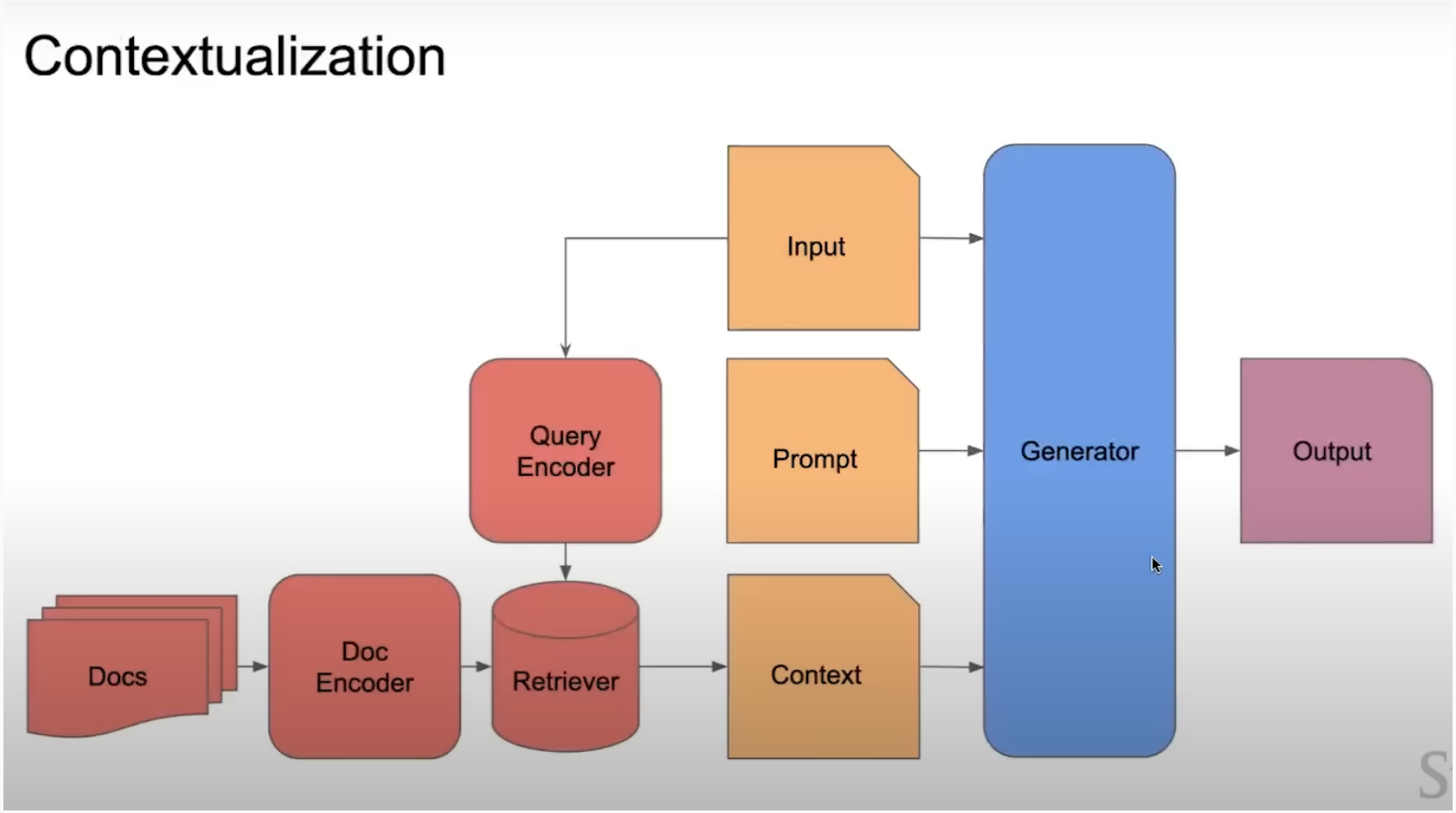
- Hallucination
- Attribution
- Staleness
- Revisions
- Customization

Solutions:

- Couple to external memory



Contextualization



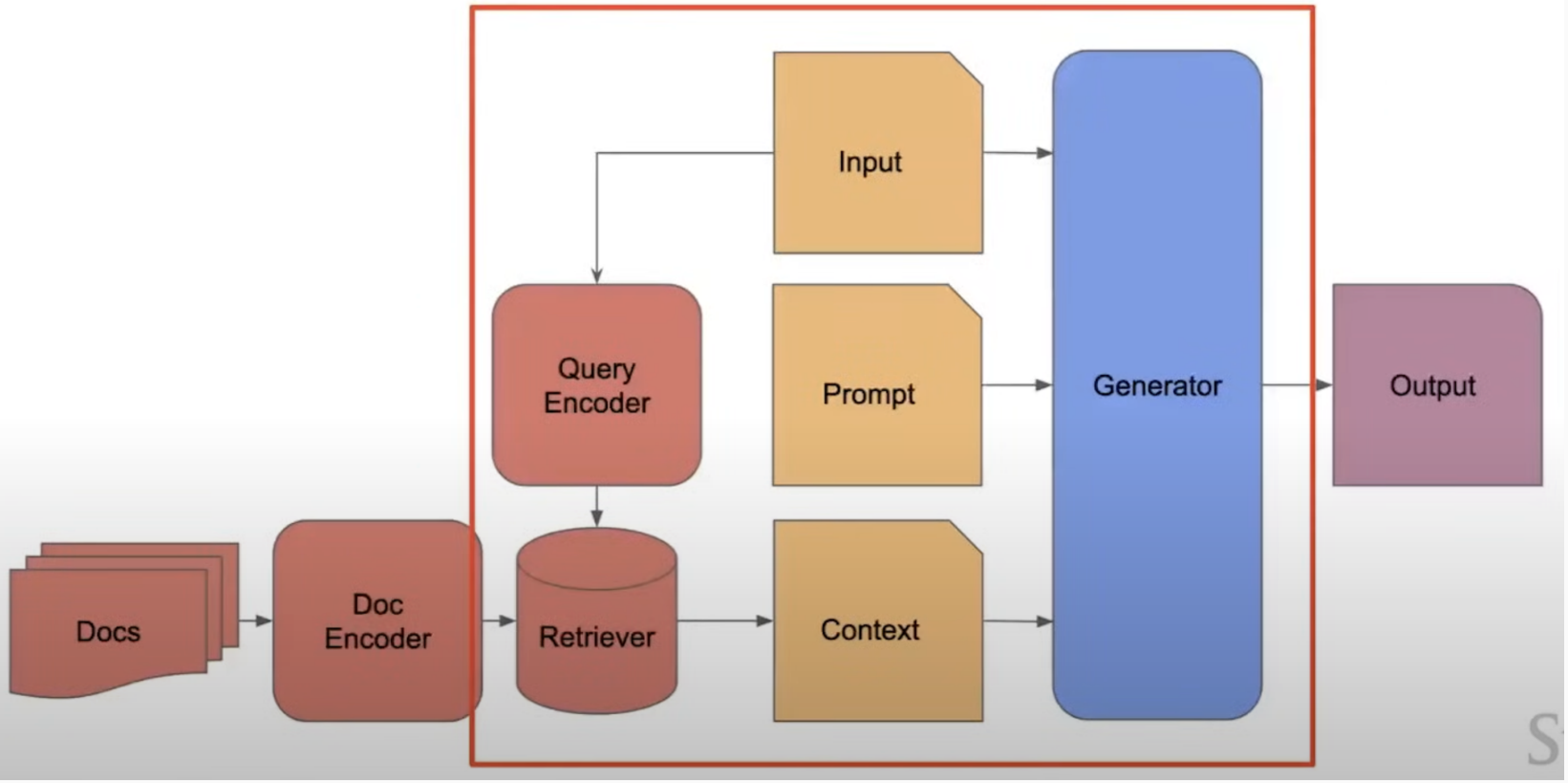
Learning Retrieval-oriented Embeddings

- Select positive and negative documents, train using a contrastive loss (e.g. hinge loss)

$$\mathcal{L}(\theta, q) = \sum_{d_{\text{pos}} \in D_{\text{pos}}} \sum_{d_{\text{neg}} \in D_{\text{neg}}} \max(0, s(q, d_{\text{neg}}; \theta) - s(q, d_{\text{pos}}; \theta))$$

- **DPR** (Karpukhin et al. 2020): learn encoders based on a BM25 hard negatives and in-batch negatives.
- **Contriever** (Izacard et al. 2022): contrastive learning using two random spans as positive pairs

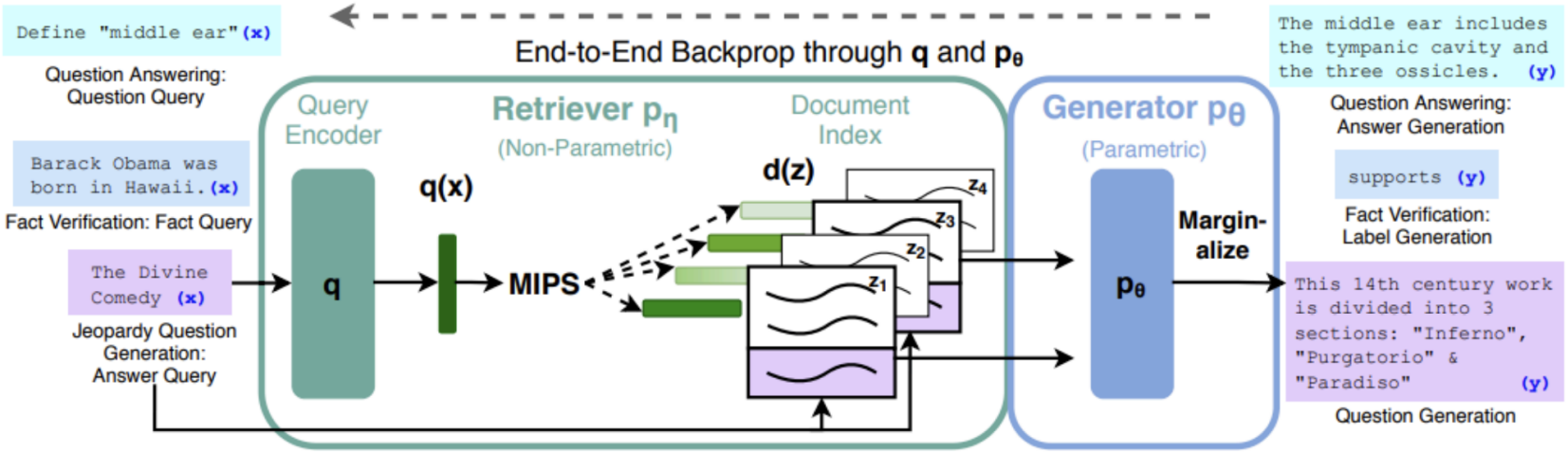
Contextualization of both



Retriever + Generator End-to-end Training ("RAG")

(Lewis et al. 2020)

- Train the retriever and reader to improve accuracy
- **Reader:** Maximize generation likelihood given single retrieved document
- **Retriever:** Maximize overall likelihood by optimizing mixture weights over documents



End-to-end Training Equations

(Lewis et al. 2020)

- Generation is a mixture model: pick a document, generate from the document

$$P_{\text{RAG}}(y|x) \approx \prod_i \sum_{z \in \text{top-k}(p(\cdot|x))} p_\eta(z|x) p_\theta(y_i|x, z, y_{1:i-1})$$

Retriever Generator

- Probability of the retriever is based on embeddings

$$p_\eta(z|x) \propto \exp(\mathbf{d}(z)^\top \mathbf{q}(x)) \quad \mathbf{d}(z) = \text{enc}_d(z), \quad \mathbf{q}(x) = \text{enc}_q(x)$$

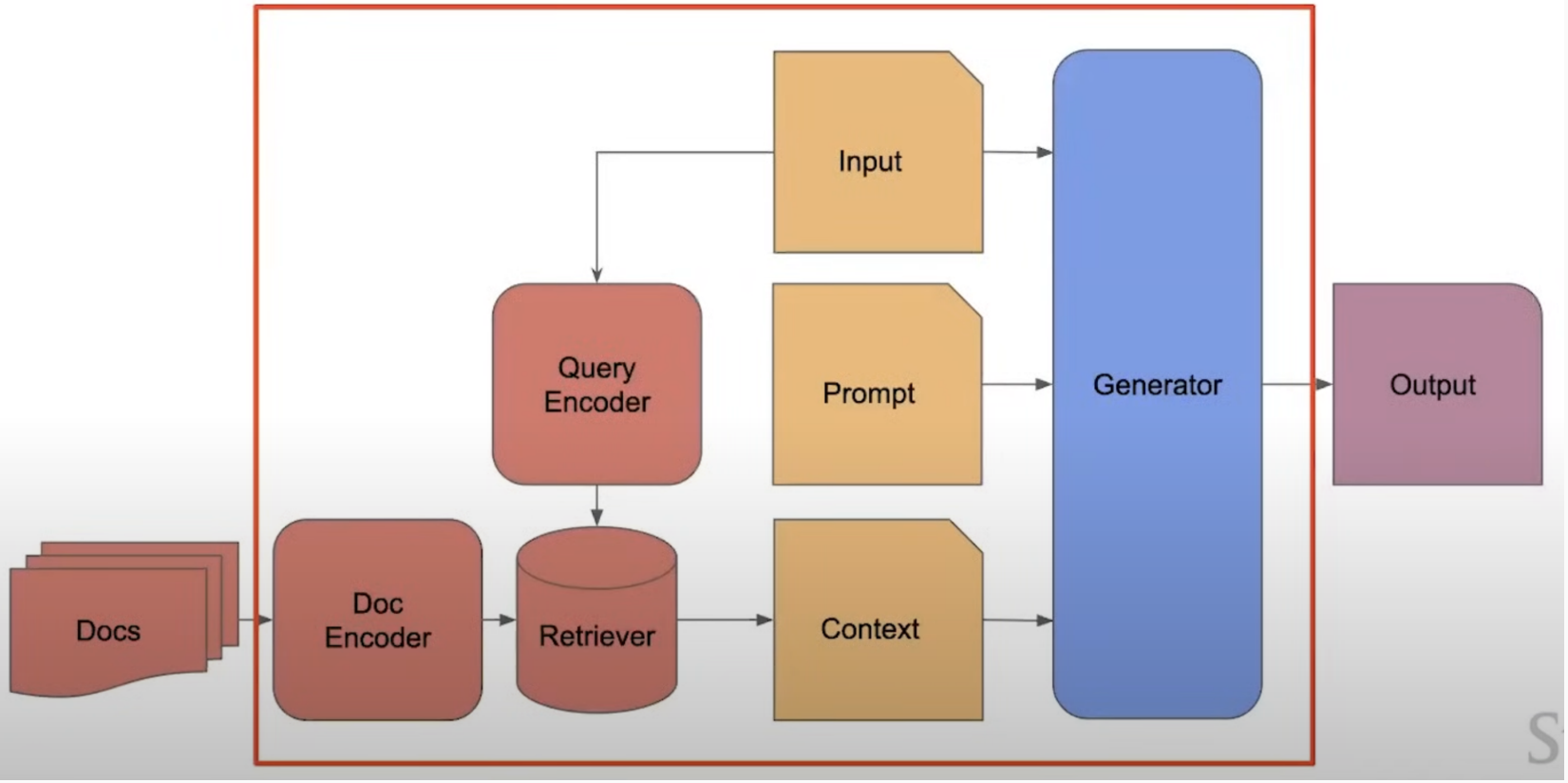
- Adjusts retriever to give higher similarities helpful docs
- Issue: search index becomes stale → can only train q(x)

Whole point of RAG: Freezing Suboptimal!

Table 6: Ablations on the dev set. As FEVER is a classification task, both RAG models are equivalent.

Model	NQ	TQA	WQ	CT	Jeopardy-QGen		MSMarco		FVR-3	FVR-2
		Exact Match			B-1	QB-1	R-L	B-1	Label Accuracy	
RAG-Token-BM25	29.7	41.5	32.1	33.1	17.5	22.3	55.5	48.4	**75.1**	**91.6**
RAG-Sequence-BM25	31.8	44.1	36.6	33.8	11.1	19.5	56.5	46.9		
RAG-Token-Frozen	37.8	50.1	37.1	51.1	16.7	21.7	55.9	49.4	72.9	89.4
RAG-Sequence-Frozen	41.2	52.1	41.8	52.6	11.8	19.6	56.7	47.3		
RAG-Token	43.5	54.8	**46.5**	51.9	**17.9**	**22.6**	56.2	**49.4**	74.5	90.6
RAG-Sequence	**44.0**	**55.8**	44.9	**53.4**	15.3	21.5	**57.2**	47.5		

Contextualization all the way



REALM (Guu et al 2020)

- The OG of non-frozen dense retrieval augmented LMs.

- Backprop all the way, async updates.
- Updating the document encoder is hard, can you see why?

- Really visionary work.
- Downside: not really generative, BERT all the way.

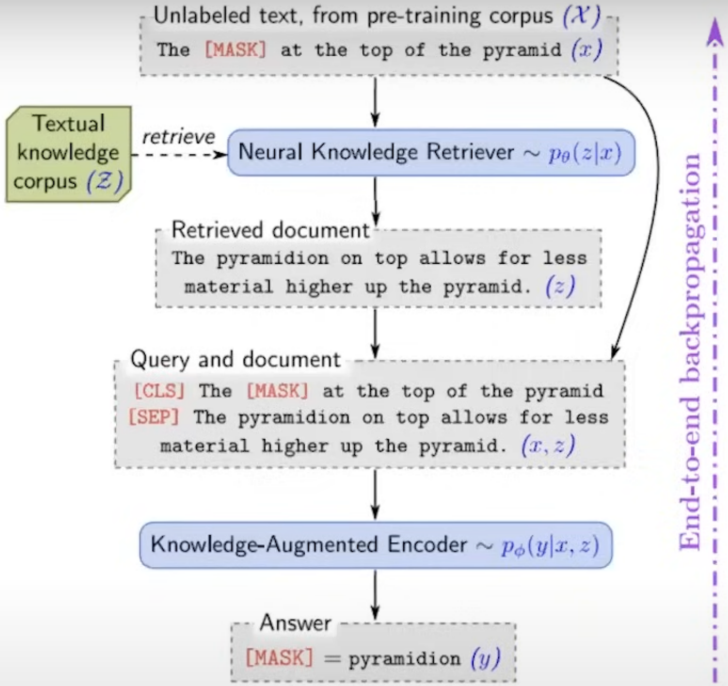


Figure 1. REALM augments language model pre-training with a **neural knowledge retriever** that retrieves knowledge from a **textual knowledge corpus**, $\mathcal{Z}$ (e.g., all of Wikipedia). Signal from the language modeling objective backpropagates all the way through the retriever, which must consider millions of documents in $\mathcal{Z}$—a significant computational challenge that we address.

When do we Retrieve?

- **Once, at the beginning of generation**
  - Default method used by most systems (Lewis et al. 2020)
- **Several times during generation, as necessary**
  - Generate a search token (Schick et al. 2023)
  - Search when the model is uncertain (Jiang et al. 2023)
- **Every token**
  - Find similar final embeddings (Khandelwal et al. 2019)

Triggering Retrieval w/ Tokens

- Toolformer (Schick et al. 2023) generates tokens that trigger retrieval (or other tools)
- Training is done in an iterative manner - generate and identify successful retrievals

The New England Journal of Medicine is a registered trademark of **[QA("Who is the publisher of The New England Journal of Medicine?") → Massachusetts Medical Society]** the MMS.

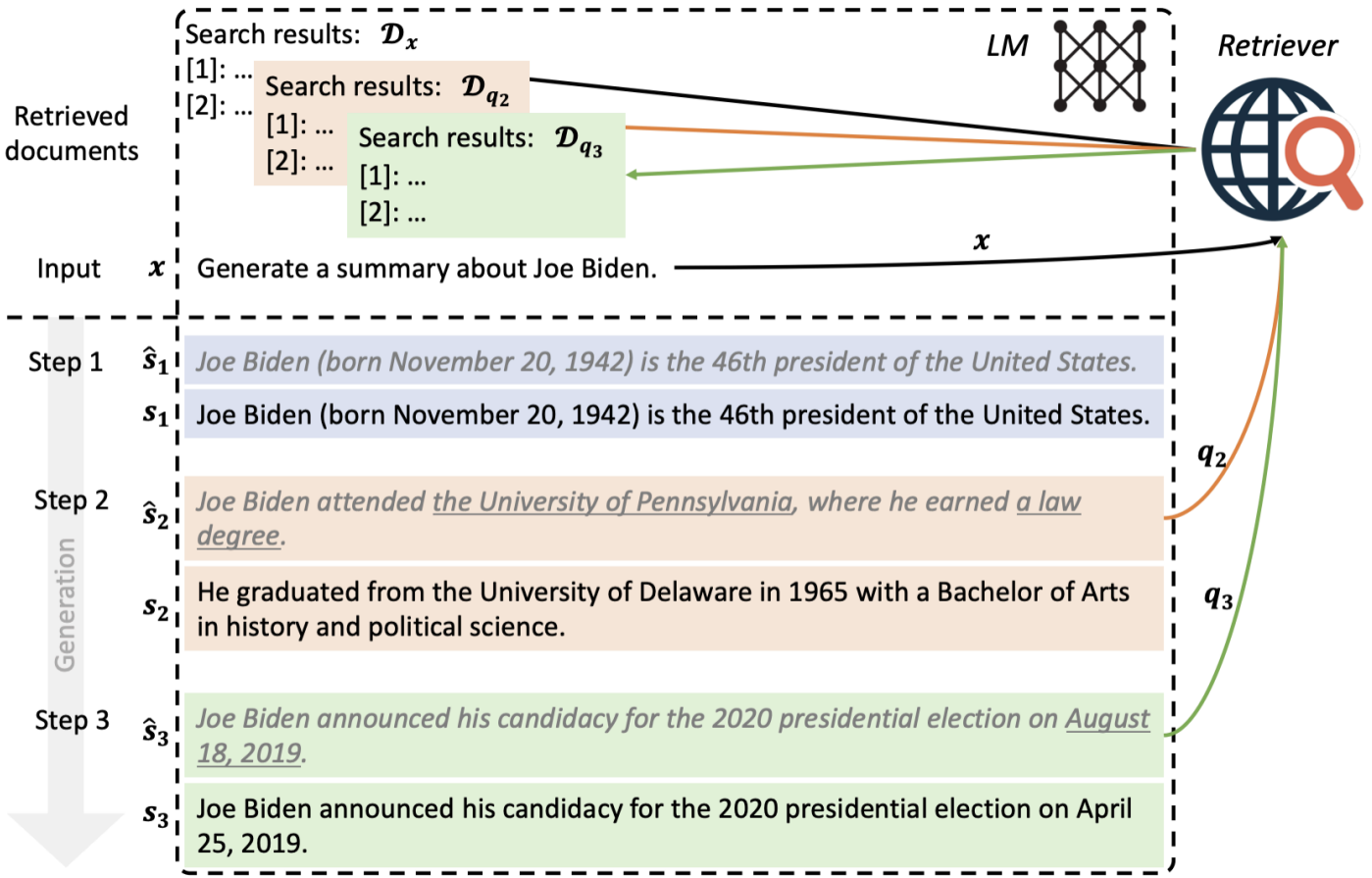
Out of 1400 participants, 400 (or **[Calculator(400 / 1400) → 0.29]** 29%) passed the test.

The name derives from "la tortuga", the Spanish word for **[MT("tortuga") → turtle]** turtle.

The Brown Act is California's law **[WikiSearch("Brown Act") → The Ralph M. Brown Act is an act of the California State Legislature that guarantees the public's right to attend and participate in meetings of local legislative bodies.]** that requires legislative bodies, like city councils, to hold their meetings open to the public.

Triggering Retrieval w/ Uncertainty

- FLARE (Jiang et al. 2023) tries to generate content, then does retrieval if LM certainty is low



Token-level Softmax Modification

- kNN-LM (Khandelwal et al. 2019) retrieves similar examples, and uses the following token from them